

MedicalAssistant 智能医疗助手

面向高并发的分布式架构优化与大模型链路治理

从单体到模块化 && Agent 调优

👤 指导老师: 贾向阳

📅 2026年05
月

👥 团队成员: 王皓霖 莫韬 陈涛 叶海峰 周锦耀



汇报大纲目录 / Menu

一、软件功能

阐述系统核心能力，介绍架构形态演进路径，覆盖单体到分布式的迭代过程，明确各阶段功能边界。

三、技术方案

落地服务治理体系，推进可靠性重构工作，叠加AI优化能力，形成完整技术落地实施方案。

二、问题分析

梳理单体架构现存痛点，拆解分布式化过程中面临的核心挑战，输出问题清单与影响范围说明。

四、测试评估

执行分层压测方案，完成量化成果对比分析，输出性能指标变化数据与效果验证结论。

系统核心功能总览



架构演进：从单体到微服务

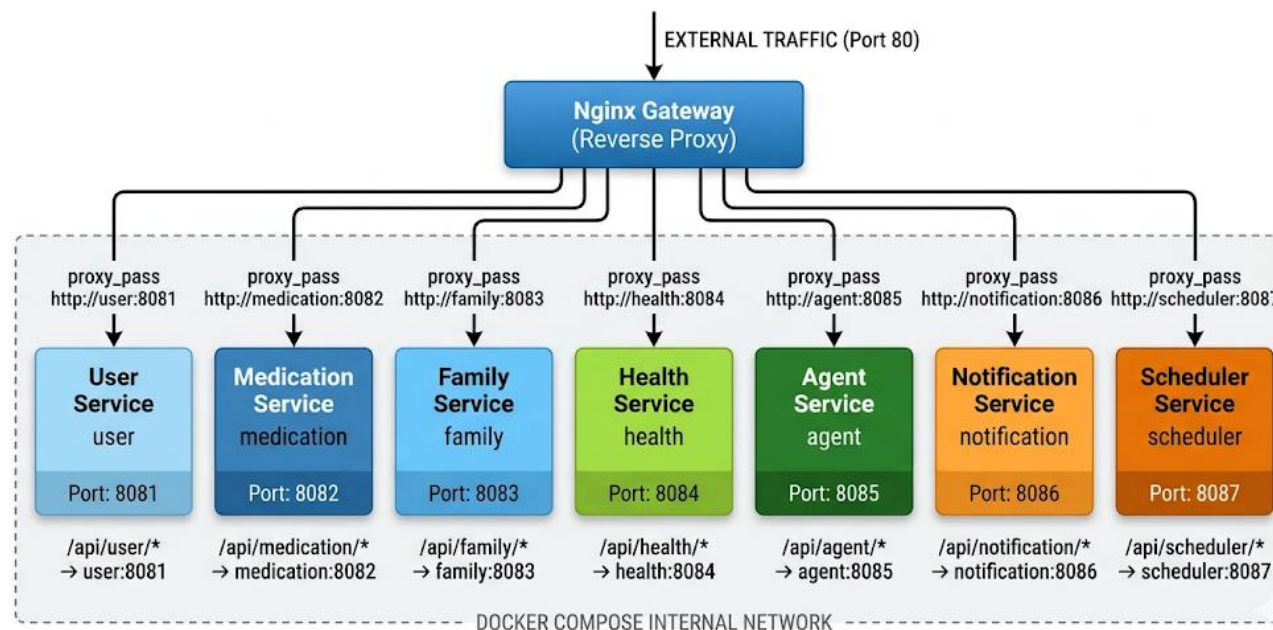
单体 Spring Boot

所有模块同 JVM，共享线程池，资源竞争严重

模块化单体

事件驱动初步引入，状态外置 Redis，Profile 开关，按领域边界拆分，Nginx 网关路由，实现高并发支持

DOCKER COMPOSE INFRASTRUCTURE: NGINX GATEWAY & MICROSERVICES ROUTING



性能与可靠性体验亮点

82.6 ×

吞吐量提升 (8→661 req/s)

100%

500 并发成功率

5.3 ×

RAG 并发吞吐提升

0次

消息与延迟任务丢失

✔ 分布式架构彻底解决了单体时代的系统性瓶颈

02 单体架构四大痛点分析

01

性能瓶颈突出

AI/OCR慢调用长期占用Web线程池资源，频繁N+1查询拖慢核心接口响应速度，影响整体服务效率。

02

可用性存在短板

WebSocket采用存单机内存模式，发生单点故障时推送服务直接中断，且无法水平扩展负载应对高并发。

03

可靠性水平较低

进程内延迟任务无内置重试机制，易导致关键业务消息出现静默丢失情况。

分布式化的核心挑战

 “拆分不是免费的”

单体旧惯例 (Local)

进程内事件 (Guava/Spring)
本机内存状态 (Session/WS)
本机定时任务 (Timer/Quartz)
本地事务与缓存 (无脑强一致)



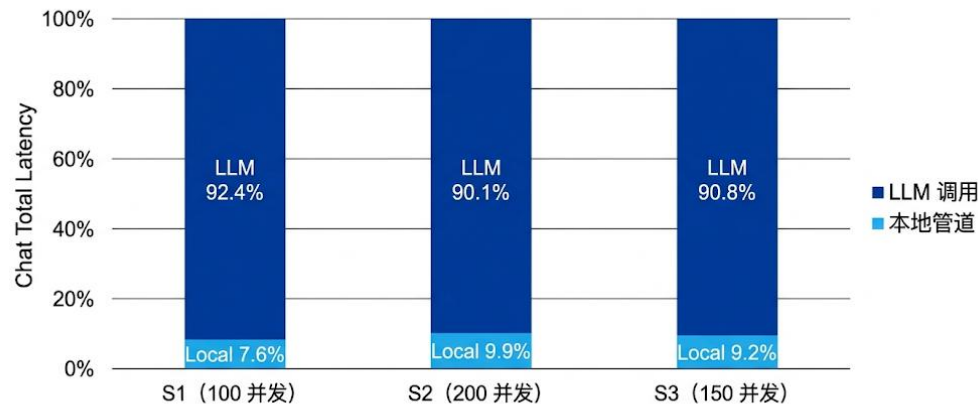
分布式新约束 (Remote)

- 需可靠消息中间件 RabbitMQ
- 状态需迁移至 Redis 分布式缓存
- 需调度器和分布式缓存协作，可靠执行任务
- 需保障幂等与最终一致性

 数据/消息一致性是重中之重（防止发布/消费丢失与无兜底）

大模型 / RAG 链路瓶颈分析

Chat 总耗时拆解：本地管道 vs LLM



真实链路中 LLM 调用占比超过 90%，本地管道 < 10%，优化重点在 AI 链路



- ✓ **链路重度依赖**：LLM 调用占 Chat 总耗时 90%+（真实 API 17s vs 本地 1.3s）
- ✓ **隐藏并发 Bug**：Flask 异步端点 + 同步阻塞 SDK 导致并发被迫串行
- ✓ **缓存误导**：原 Benchmark 命中缓存（5ms），掩盖了 Flask 的卡死缺陷

 **结论**：分层压测是发现长链路隐蔽缺陷的唯一手段



“有理有据”：架构拆分依据与技术取舍

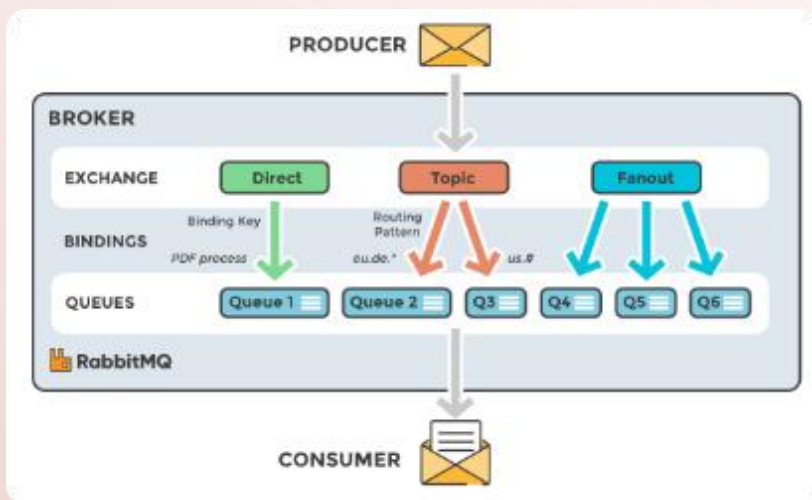
田 边界划分原则

- 隔离核心业务：登录/健康不被 AI/PDF 慢调用拖慢
- WebSocket 服务：单独 notification-service 模块维护，配合 RabbitMQ 幂等性处理消息、消息处理重试
- 延迟任务集中：仅由 schedule-service 消费
- Service 职责：仅发布事件，不做复杂副作用

• 选型做减法

技术	为什么不用？(方案判断力)
Nacos	模块化单体下 Nginx 静态路由足够，降低运维复杂度
分库	当前数据量无需分库，避免跨库查询难度
OpenFeign	保留直接调用，性能更优，调试极简

事件消息执行三重保障



发布确认机制

采用Publisher Confirm与Returns机制，保障消息成功送达交换机与队列，投递成功率可提升至99.9%以上。

消费确认模式

启用Consumer Manual ACK模式，待业务逻辑执行完成后再进行消息应答，避免消息丢失问题。

死信兜底方案

配置DLQ死信队列，消息重试3次耗尽后自动入队，支持后续人工追溯核查异常消息。



分布式延迟任务可靠性

维度	改进前 (JVM 单机)	改进后 (Redis + Lua)
存储机制	JVM 堆内存 Map	Redis ZSET (infra:delay:tasks) 持久化
多实例	各节点重复执行	Lua 原子 POP (ZRANGE + ZREM) 互斥抢占
重启恢复	丢失或全表重扫	毫秒级冷启动恢复，数据零丢失
重试策略	无，失败即丢	指数退避 (2s→4s→8s) + 最终入 DLQ



技术方案服务治理与全链路可观测

01

限流降级管控

采用Sentinel实现接口限流降级，覆盖
/agent/chat/stream、
/api/ocr/*、
/agent/chat三类核心接口，保障服务稳定性。

02

链路追踪配置

基于Zipkin+Brave
5.16+Micrometer构建
链路追踪体系，开发环
境采样率100%，生产环
境采样率10%。

03

追踪性能开销

链路追踪在生产环境采
样率控制在10%，可将
额外性能开销控制在5%
以内，平衡观测与性能。

大模型 / RAG 治理优化历程

迭代一：可靠性

三层 LLM 回退 (MedicalAgent → Streaming → Block) + 空响应重试

迭代二：经济性

ChatCacheService 基于 MD5 缓存结果, 短文本 TTL 300s 大幅降本

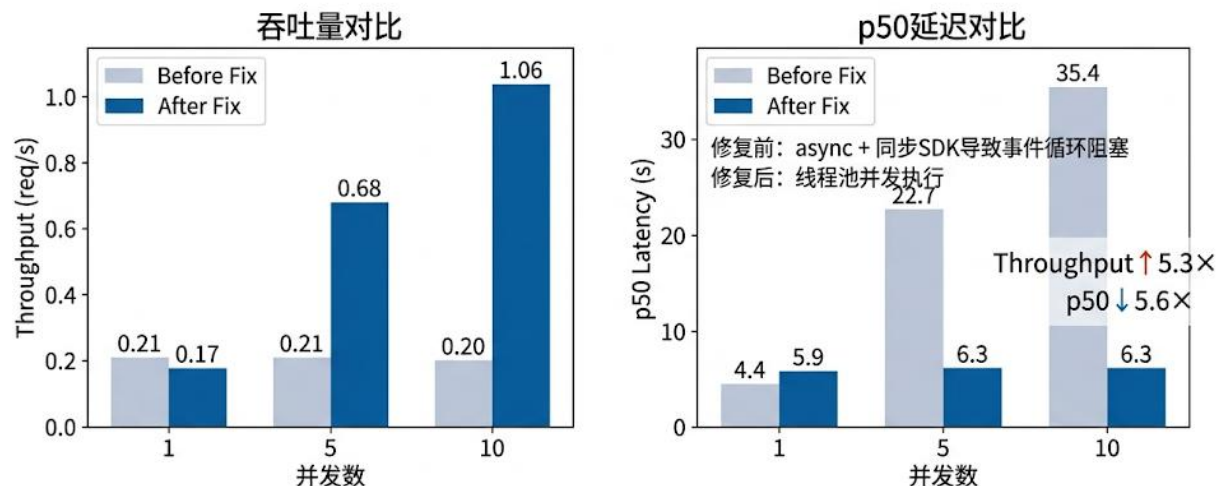
迭代三：并发优化

修护 Flask 并发串行 Bug; aiExecutor 线程池动态扩容至 200

迭代四：RAG 治理

连接池统一、Redis 缓存锁防击穿、Bulkhead 限流保护向量库

大模型/RAG并发修复效果



修复前并发无法提升吞吐, 延迟随并发快速增长; 修复后吞吐近线性增长, p50稳定在约6秒。



分层多场景系统化压测方案

场景矩阵压测方案

场景	并发线程 (t)	压测目标
S1 基线	100t	正常负载验证
S2 高并发	200t	系统性能峰值
S3 长稳	150t × 10min	内存泄漏与稳定性
S4 毛刺	300t	瞬时过载处理
S5 过载	500t	降级/熔断兜底

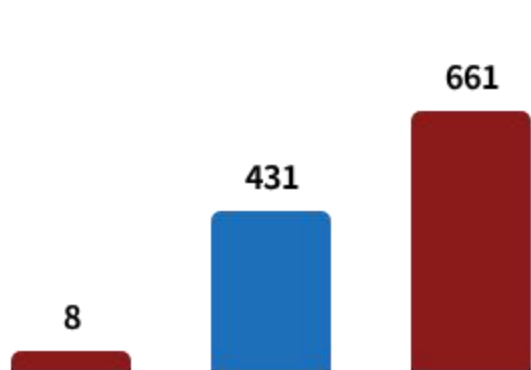
工具: JMeter (SSE TTFC), Python aiohttp

双模式 & 分层对照示意

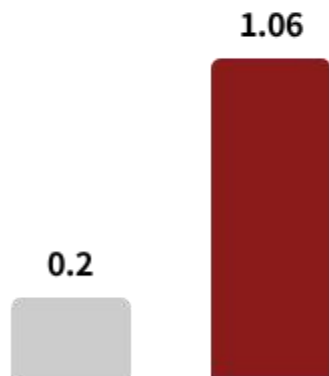


分层: Spring 端到端 (经缓存) ∨ Flask 直调 (cache-miss)

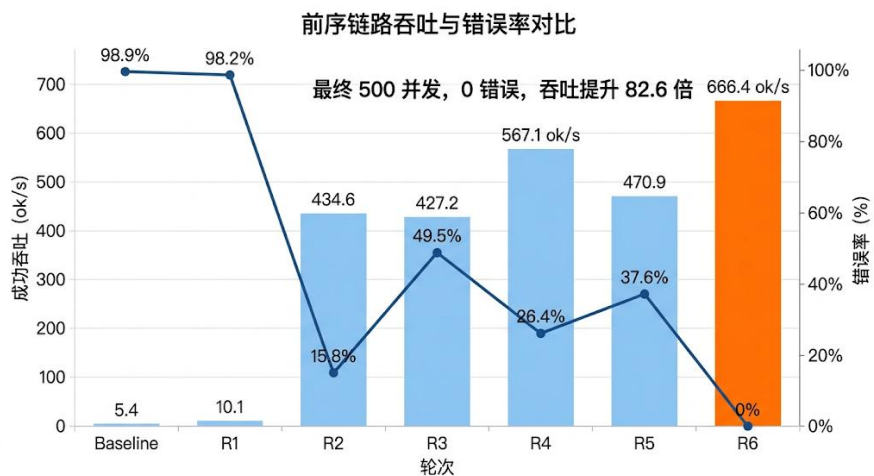
量化成果对比：用数据说话



吞吐量 (Baseline → R6) 提升 82.6×



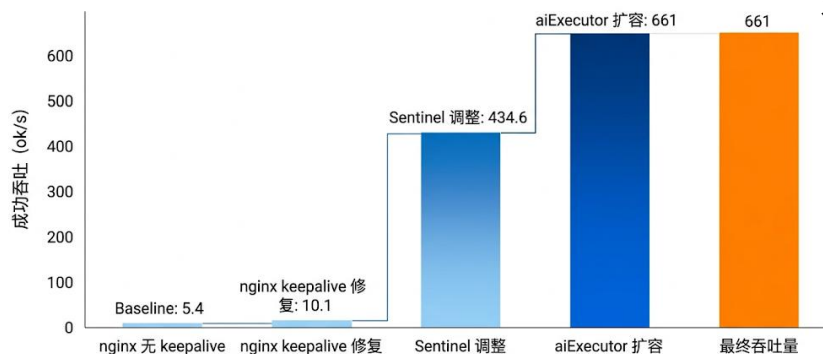
RAG 并发修复 (10 并发) 提升 5.3×



前序链路吞吐与错误率对比

最终 500 并发, 0 错误, 吞吐提升 82.6 倍

前序链路瓶颈消除吞吐跃升



每消除一个瓶颈, 吞吐跃升明显

- ✓ **高可用性**: 500 并发下 100% 成功率, 零网关报错
- ✓ **接口覆盖**: 45 个端点 93.3% 测试通过率
- ✓ **解耦代价**: RabbitMQ 带来 170-200ms 延迟增量 (可接受)
- ✓ **链路追踪**: 全链路 Trace 注入日志, 系统开销 < 5%

汇报结束，谢谢聆听

核心总结

- ✓ 分布式重构，支撑 82 × 吞吐跨越
- ✓ 事件驱动与 Redis 持久化保障可靠性
- ✓ 大模型长链路深度治理与并发修复

未来展望

- ✓ 引入 Outbox 模式保障强一致性
- ✓ 生产环境采样率动态调优
- ✓ 多 LLM API Key 轮询负载均衡

请老师批评指正